Motore grafico e viewer 3D

Progetto software in UML (Series 4), versione semplice · Computer Graphics M-I5203, SUPSI · Alessio Gervasini · ottobre 2026

> **Milestone Series 4.** Il codice degli esercizi diventa una libreria con un'API: classi per mesh, luci, materiali, camere. L'applicazione usa solo queste classi e non chiama mai OpenGL o FreeGLUT. Questo documento mostra il progetto con quattro diagrammi semplici e spiega quale modello 3D useremo e perché.

1. Architettura

Due programmi: il **motore** (`engine.dll`) e il **viewer** (`client.exe`). Il viewer parla solo con l'API `Eng::`. OpenGL, FreeGLUT e Assimp restano dentro il motore; GLM invece fa parte dell'API (Series 1), perché le funzioni pubbliche usano `glm::vec3` e `glm::mat4`.

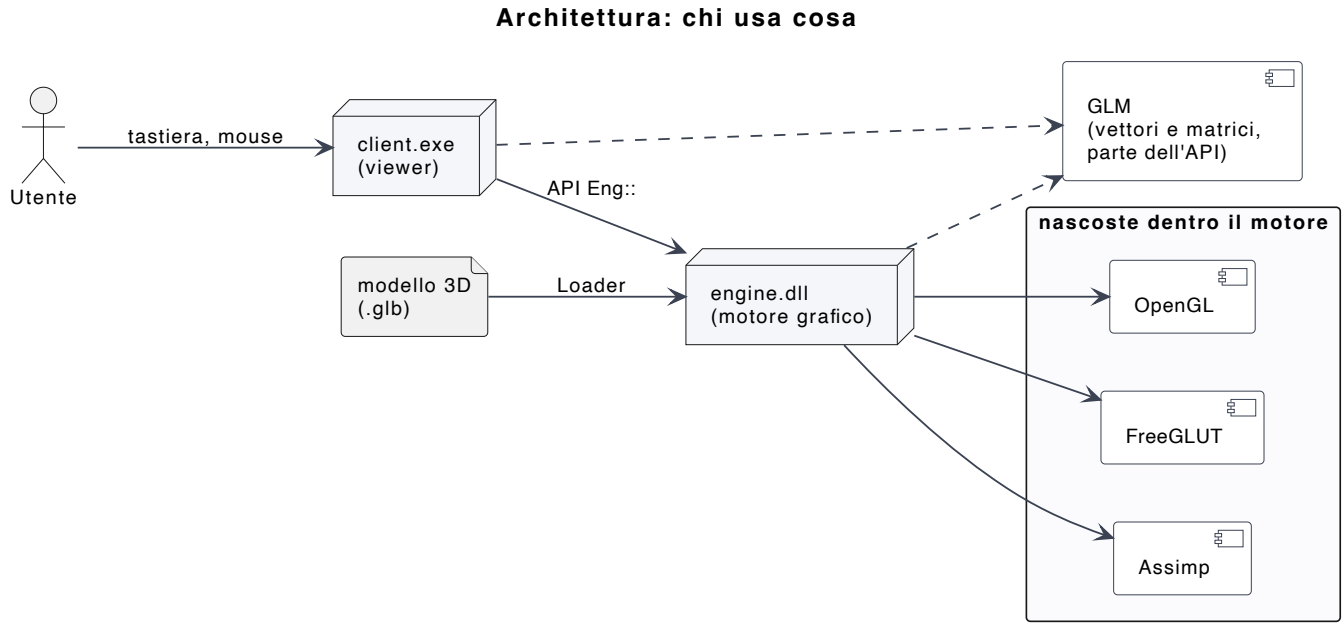


Figura 1 – Componenti e dipendenze.

2. Classi

- **Base**: la classe principale, un singleton (una sola finestra). Interfaccia della Series 3: `init`, `clearWindow`, `render`, `swapBuffers`, `free`.
- **Object**: radice comune, con id, nome e `render()`.
- **Node**: elemento dello scene graph (matrice + figli). Mesh, Camera e Light sono nodi.
- **Light**: astratta, con due tipi: **DirectionalLight** e **PointLight**.
- **Material** e **Texture**: aspetto delle superfici.
- **List**: lista delle cose da disegnare (prima le luci, poi le mesh).
- **Loader**: legge il file 3D e crea lo scene graph.

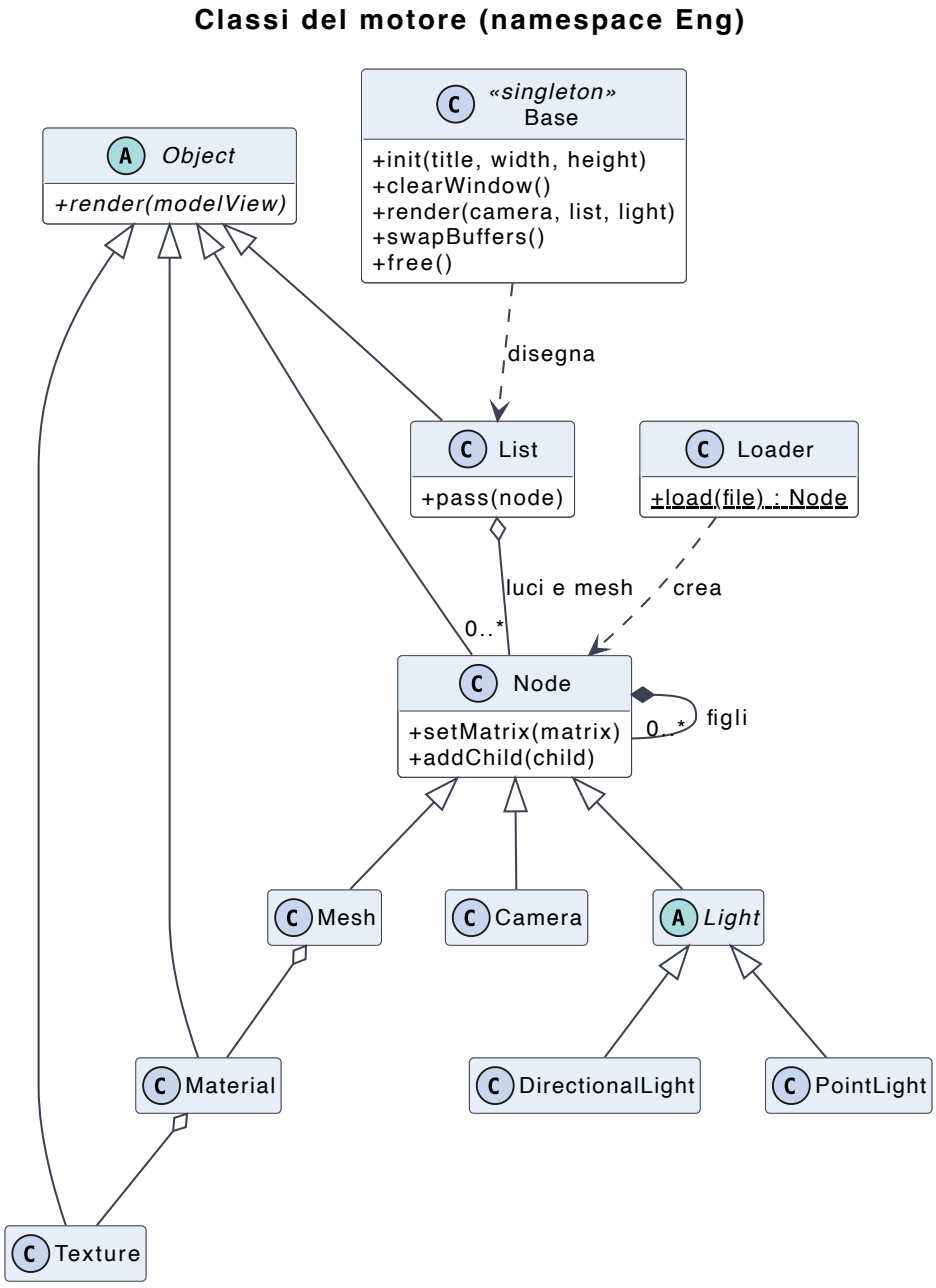


Figura 2 – Classi del motore e loro relazioni.

3. Ciclo di vita del viewer

Lo stesso schema del `main()` richiesto dalla Series 3: inizializza, ciclo *pulisci → disegna → scambia i buffer*, chiudi.

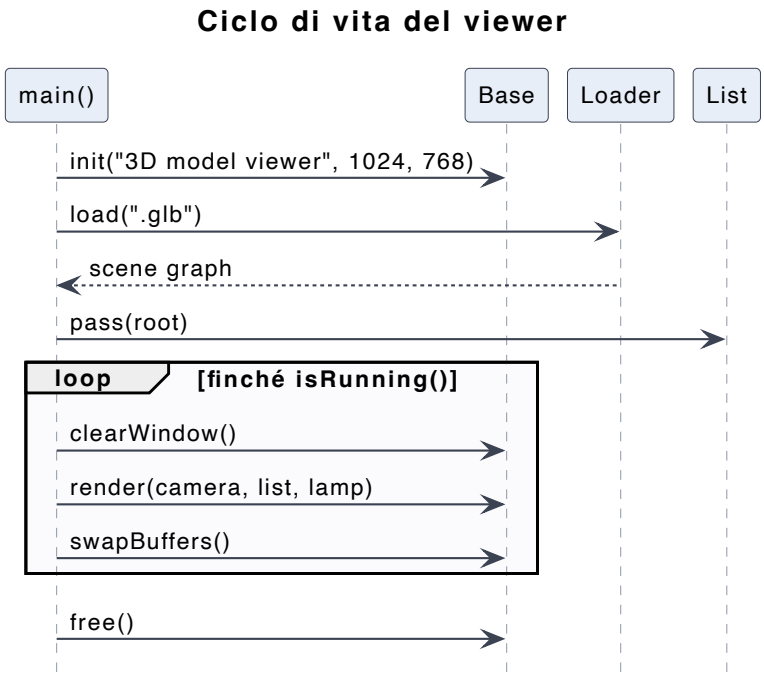


Figura 3 – Sequenza principale del viewer.

4. Scene graph del viewer

Il Loader crea il ramo *modello*; il viewer aggiunge le due luci e le tre camere.

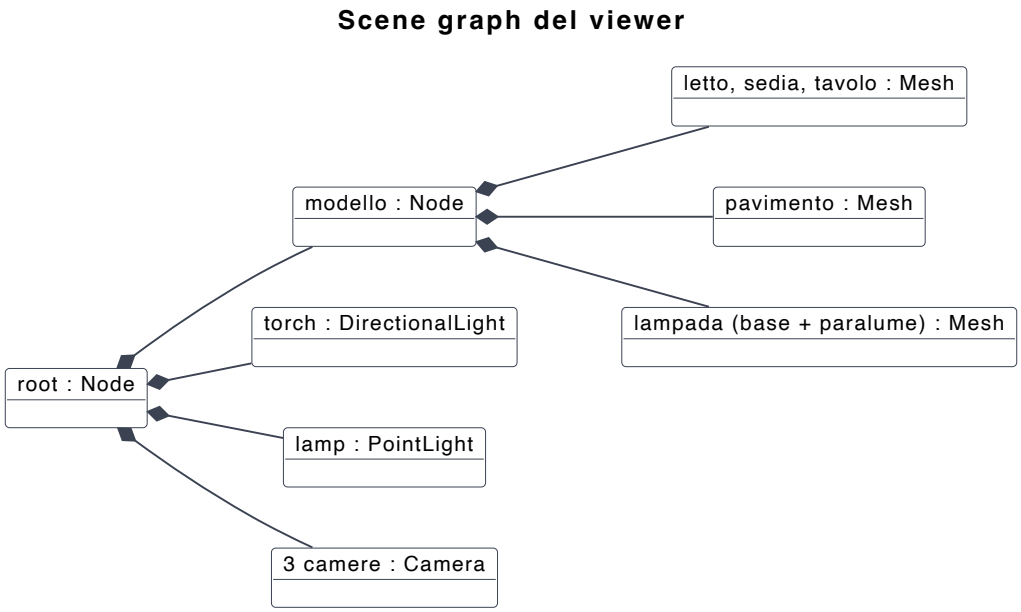


Figura 4 – Oggetti a runtime (semplificato).

5. Il modello 3D che useremo

“Bedroom Furniture Set – Game Ready”: letto, scrivania e sedia su un pavimento, a cui abbiamo aggiunto una lampada da scrivania con il paralume luminoso. Le immagini sono catturate dal nostro viewer.



Figura 5 – Camera frontale, torcia e lampada accese.



Figura 6 – Camera dall'alto.



Figura 7 – Torcia spenta: illumina solo la lampada, che proietta l'ombra sul pavimento.

Dato	Valore
Mesh	4 (letto + mobili, pavimento, base e paralume della lampada)
Triangoli	14 837, molto sotto il limite di 100k del requisito sui 30 FPS
Materiali e texture	4 materiali, 4 immagini PNG dentro il file
Dimensioni	circa 4 × 1,3 × 4 m (scala reale)
Autore e licenza	General of Thailand (sketchfab.com/Melonpolygons), CC BY 4.0, modificato

Perché questo oggetto

- **Pavimento piatto**: superficie su cui proiettare l'ombra (requisito “ombre o riflessi su un piano”).
- **Lampada**: posto naturale per la luce statica puntiforme; la luce dinamica direzionale è la torcia che segue la camera.
- **Texture vere** (coperta, legno): si vede subito se il texture mapping funziona.
- **Pochi triangoli**: il viewer resta fluido anche su PC senza scheda video potente.

6. Perché il formato .glb

Il requisito del viewer (cg_introduction, slide 33) chiede di caricare da file esterno un modello **con materiali, luci e texture**. Il formato deve quindi portare più della sola geometria.

Formato	Geometria	Materiali	Texture	Gerarchia di nodi	Luci	Note
.glb (glTF 2.0)	**sì**	**sì**	**sì, dentro il file**	**sì**	**sì (estensione standard)**	standard aperto Khronos, un solo file
.ply	**sì**	**no**	**no standard**	**no**	**no**	pensato per scansioni 3D (nuvole di punti, colori per vertice)
.obj + .mtl	**sì**	**sì**	file separati	**no**	**no**	semplice, ma niente nodi né matrici
.fbx	**sì**	**sì**	**sì**	**sì**	**sì**	formato proprietario Autodesk

- **Perché non .ply.** Un file PLY contiene vertici, facce e al massimo colori o normali per vertice: niente materiali, niente immagini di texture, niente nodi con matrici, niente luci. Con un PLY il requisito non si potrebbe soddisfare.
- **Un solo file.** Il .glb contiene geometria, normali, coordinate texture, materiali e le immagini PNG. Niente percorsi da sistemare quando si copia la demo su un altro PC.
- **Gerarchia = scene graph.** I nodi del file hanno una matrice locale e dei figli, come il nostro `Node`: il Loader copia l'albero così com'è.
- **Standard aperto** (Khronos, lo stesso gruppo di OpenGL). Blender lo apre e lo salva senza plugin (il corso ammette Blender al posto di 3ds Max) e Sketchfab lo offre come download.
- **Lettura con Assimp.** Il Loader converte i dati di Assimp nelle nostre classi; le texture si decodificano dalla memoria con `stb_image`.

> **Da completare.** Il file oggi non contiene luci: le due luci le crea il viewer (la lampada viene posizionata sul paralume trovato dal suo materiale luminoso). Prossimo passo: aggiungere la luce della lampada al .glb e farla leggere al Loader, così anche le luci arrivano dal file come chiede il requisito.